

MULUNGUSHI UNIVERSITY

School of Engineering and Technology

Department of Computer Science and IT

ICT461 – Web Standards and HTTP Fundamentals

Task 3 Report: Task 3.2 — CORS and Browser Boundaries

Kondwani nyoni 202205306

3. Task 3.2 — CORS and Browser Boundaries

The interface (<http://127.0.0.1:5500>) and the API (<http://localhost:3000>) run on different origins (different ports), so any Fetch request from the browser to the API is cross-origin. This section shows the request failing before CORS was configured, the OPTIONS preflight, and the request succeeding once the interface origin was allowed.

3.1 Before CORS was configured

With no allowed-origins list on the server, a GET `/api/courses` call from the page failed. The browser blocked the response because it carried no `Access-Control-Allow-Origin` header, and the console reported both the CORS failure and an underlying 500 from the server's origin-check callback rejecting the request.

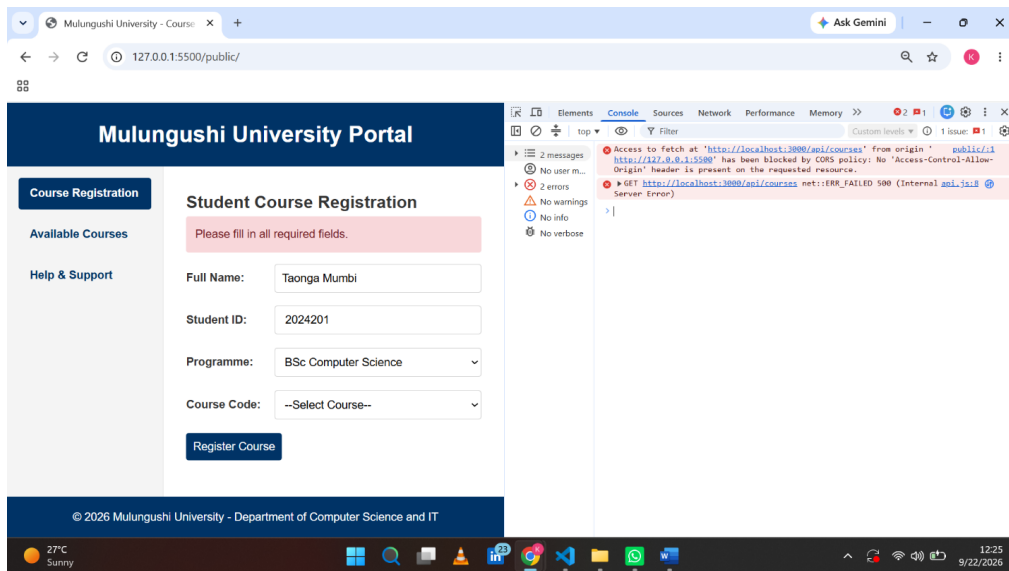


Figure 4: Console shows "No 'Access-Control-Allow-Origin' header is present" and a 500 (Internal Server Error) for GET `/api/courses`, called from origin <http://127.0.0.1:5500>.

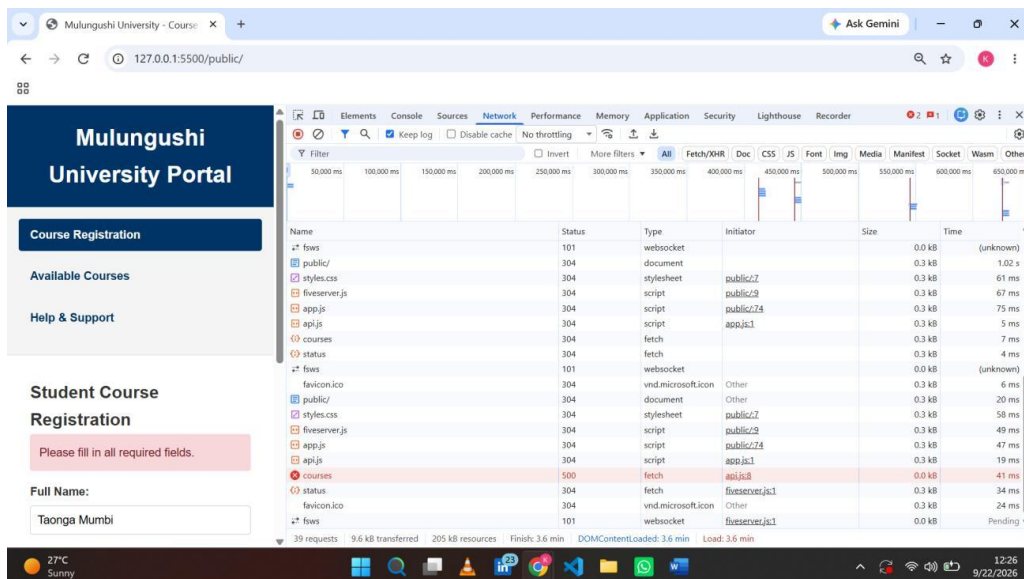


Figure 5: Network waterfall confirming the same courses request completed with status 500 while all same-origin static assets (`index.html`, `styles.css`, `app.js`, `api.js`) loaded normally.

3.2 Confirming the origin check on the server (cURL)

Because cURL is not a browser, it does not enforce CORS — it was used here to test the server's own origin-allow logic directly, independent of the browser's same-origin policy.

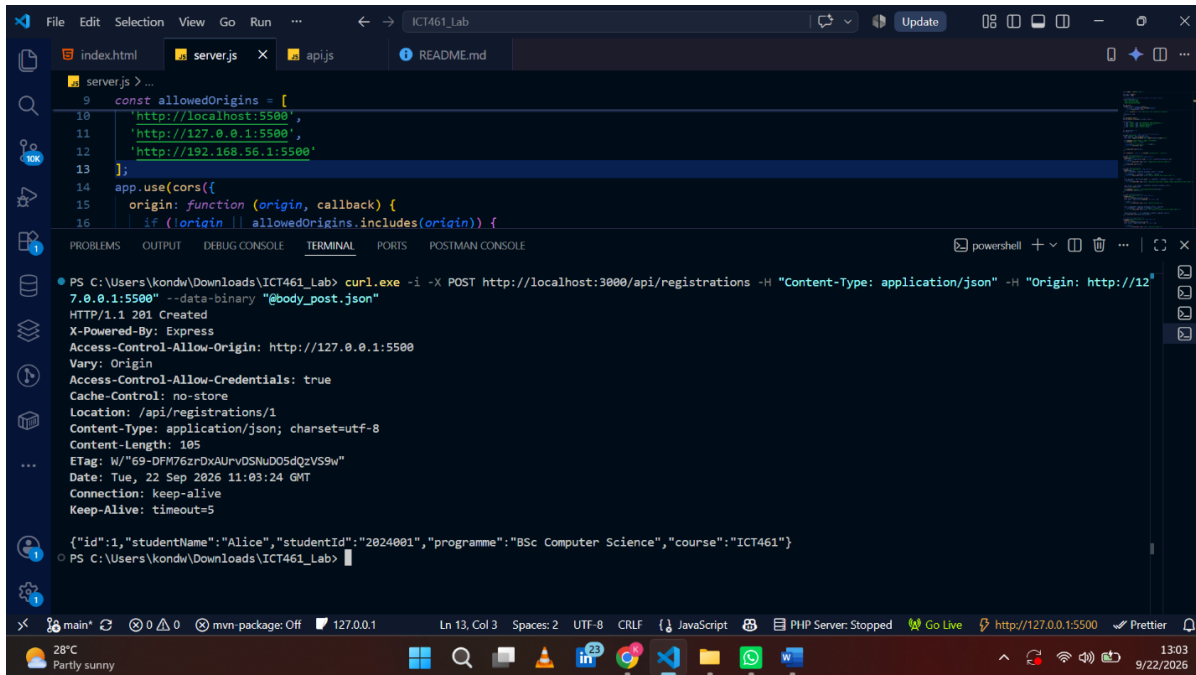


Figure 6: POST /api/registrations from cURL with Origin: http://127.0.0.1:5500 (an allowed origin) succeeds — 201 Created, Access-Control-Allow-Origin echoes the request origin, and Location: /api/registrations/1 is returned.

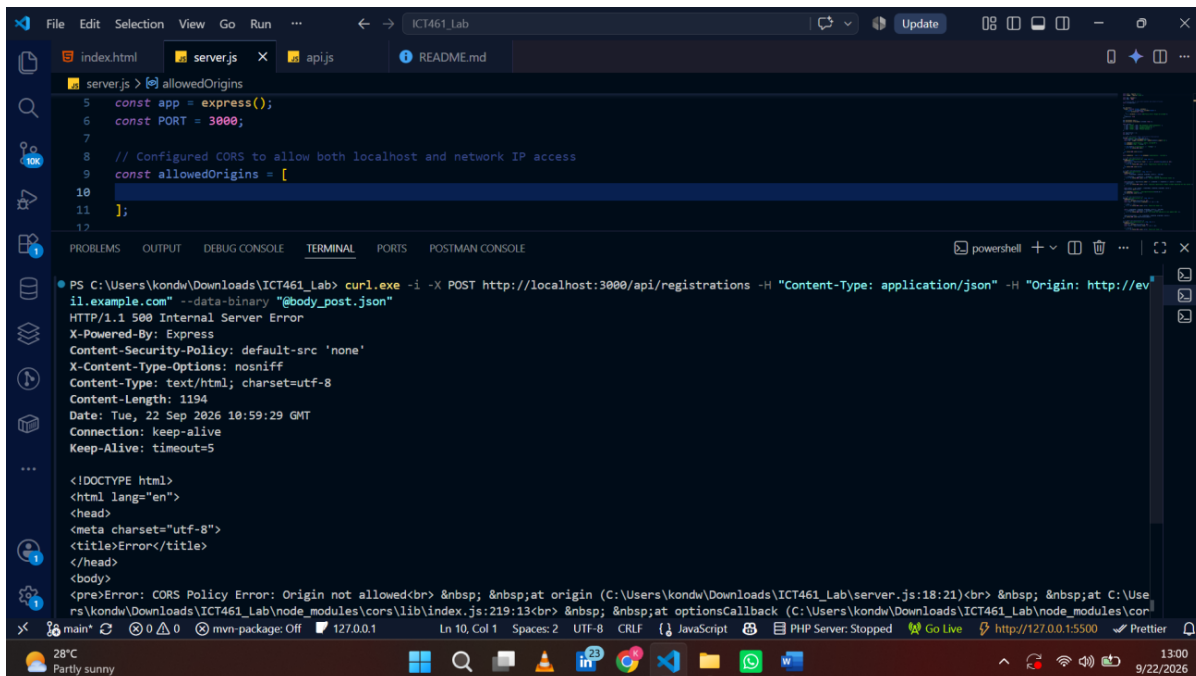


Figure 7: The same POST with Origin: `http://evil.example.com` (not in the allow-list) is rejected by the server's own `origin()` callback with "CORS Policy Error: Origin not allowed" (500) — proving the check happens server-side, not only in the browser.

3.3 Browser preflight before the interface origin was allowed

Submitting the registration form from the page triggers a JSON POST, which is a "non-simple" request and so the browser first sends an automatic OPTIONS preflight. Before `http://127.0.0.1:5500` was added to the allow-list, the preflight failed.

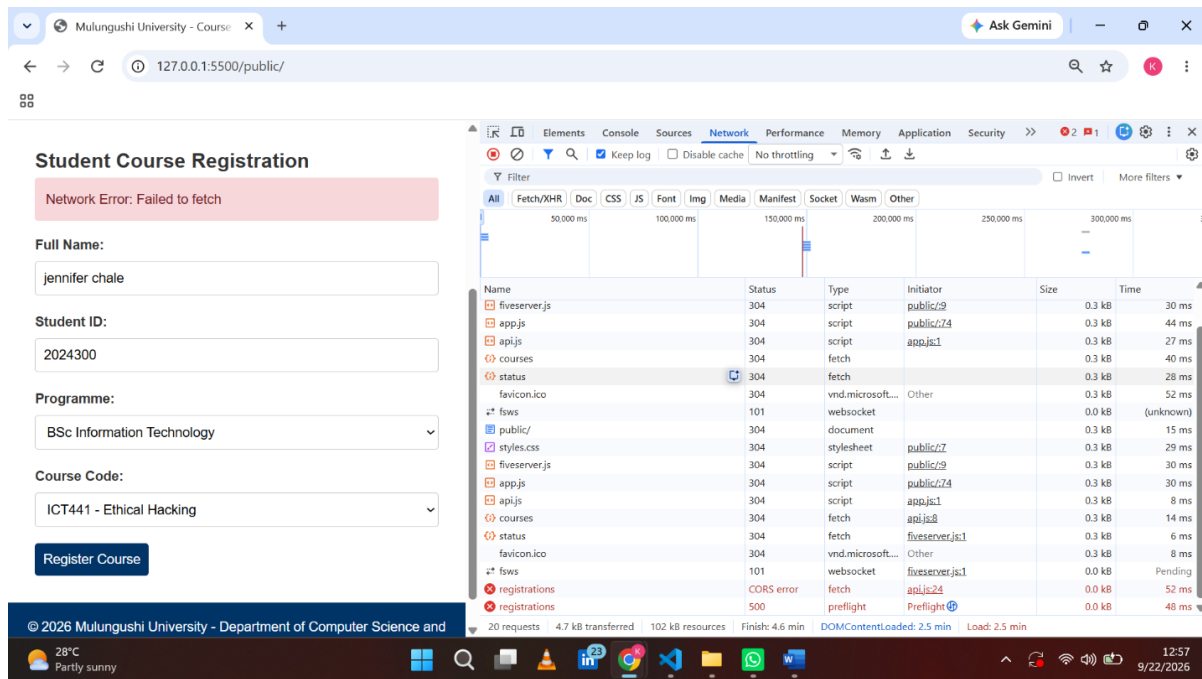


Figure 8: Network panel shows the registrations request marked "CORS error" and the preceding OPTIONS preflight returning 500; the form displays "Network Error: Failed to fetch".

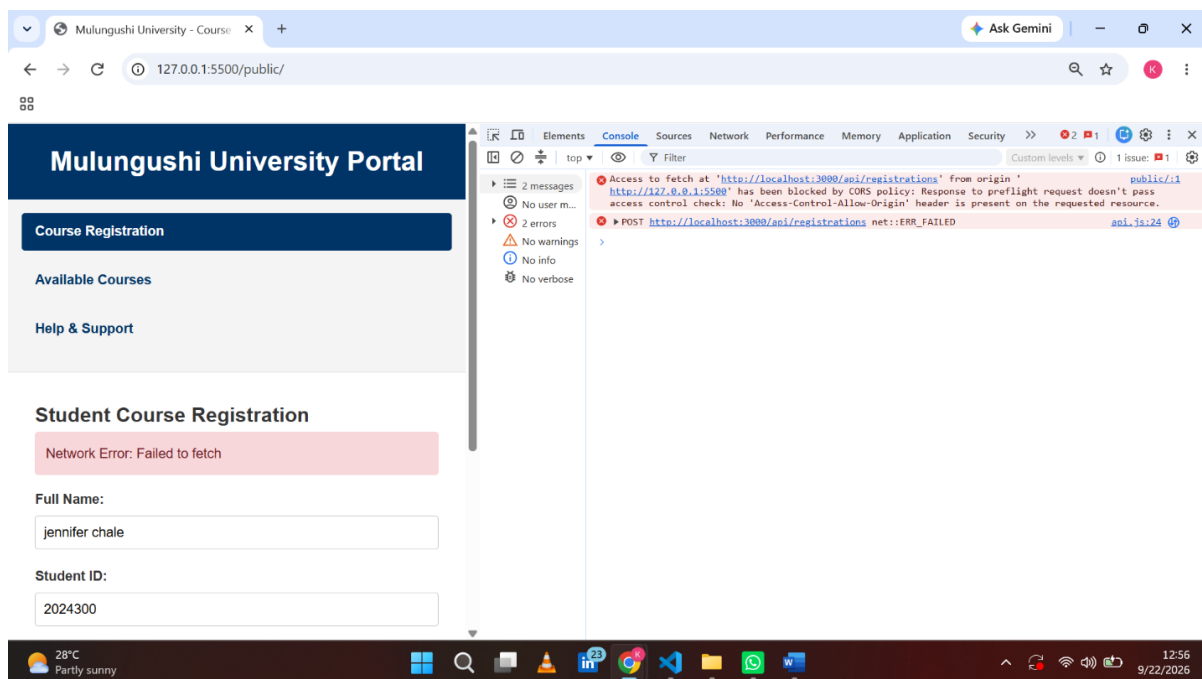


Figure 9: Console detail — "Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource", confirming the preflight, not the actual POST, is what failed.

3.4 After allowing the interface origin

server.js was updated to allow `http://localhost:5500`, `http://127.0.0.1:5500` and `http://192.168.56.1:5500` explicitly, alongside the required methods and the Content-Type header. The registration form was resubmitted with the same data.

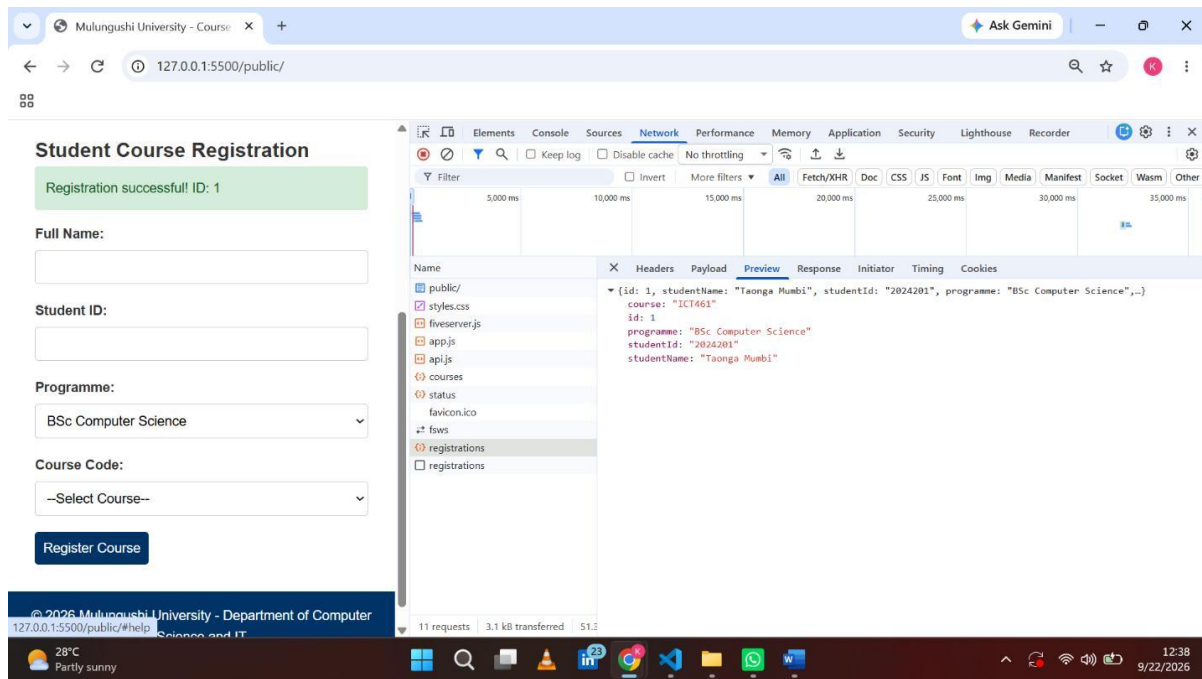


Figure 10: With CORS correctly configured, the browser POST succeeds — the form shows "Registration successful! ID: 1" and the Network preview shows the created record returned by the server.

3.5 Browser vs cURL — comparison

- cURL has no concept of "origin" restrictions on the response side: it will read whatever the server sends back, whether or not an Access-Control-Allow-Origin header is present. This makes it useful for testing the server's own logic in isolation (Figures 6–7).
- A browser enforces the same-origin policy on the response before handing it to page JavaScript, and for non-simple requests it inserts an OPTIONS preflight the developer never writes explicitly. Both the preflight and the actual response must carry a matching Access-Control-Allow-Origin (and, for credentialed requests, Access-Control-Allow-Credentials: true) or the fetch() call rejects with a network-level error rather than a normal HTTP status the application code can branch on.

Browser security (the same-origin policy and CORS enforcement) was left enabled throughout; every fix was made by changing server headers/allow-lists, not by disabling checks in the browser.

4. Predicted vs Actual Results (Checkpoint B Log)

Step	Predicted Result	Actual Result	Evidence	Explanation
Unconfigured CORS: GET /api/courses from page	Request blocked by the browser; no data rendered.	Console: no Access-Control-Allow-Origin header; request shows 500.	Figures 4–5	Server had no allow-list yet, so its own origin callback rejected the request before CORS headers could be added.
cURL POST with allowed Origin header	201 Created, since cURL is not itself blocked by CORS.	201 Created; Access-Control-Allow-Origin echoes 127.0.0.1:5500; Location header present.	Figure 6	Confirms the server-side allow-list logic works for a permitted origin.
cURL POST with disallowed Origin header	Rejected by the server's own origin check.	500 "CORS Policy Error: Origin not allowed".	Figure 7	Shows the restriction is enforced server-side, not only by the browser.
Browser POST before interface origin allowed	OPTIONS preflight fails; form shows a network error.	Preflight returns 500; console reports missing Access-Control-Allow-Origin; form shows "Network Error: Failed to fetch".	Figures 8–9	Non-simple JSON POST triggers a preflight; it must succeed before the browser will even send the real request.
Browser POST after CORS fix	201 Created; form confirms success.	"Registration successful! ID: 1"; response body visible in Network preview.	Figure 10	Allow-list plus allowed headers/methods resolved the preflight, and the real POST then completed normally.

5. Conclusion

Task 3 demonstrated two related but distinct browser/server boundaries. Caching (ETag + Cache-Control) lets a client avoid re-fetching unchanged data while still guaranteeing it sees new data as soon as the resource changes, and no-store was used to ensure registration data is never served stale. CORS enforces that only explicitly trusted origins can read cross-origin API responses from a browser context; cURL bypasses this enforcement entirely, which made it a useful tool for isolating server-side origin logic from browser-side preflight behaviour. In both areas, the fixes were made entirely through server response headers, without disabling any browser protections.